

1.2 Variables sur la stack

Q1: Les adresses a,b,c sont consécutives en mémoire, chaque adresse à 4 octets de différence car c'est la mémoire allouée à un int en stack.

Q2: Le stack grandit vers des valeurs mémoire décroissante

Q3: La variable a est effacé quand la fonction afficher se termine.

2.3 Détecter avec les sanitizers

Q1: Le programme A a un heap-buffer-overflow et le sanitizer nous montre l'adresse Mémoire.

Partie 5 -- Bilan : tableau récapitulatif

Concept	Définition	Exemple en C/C++
Stack(pile)	Mémoire petite, est gérée automatiquement	<code>int num = 0; char str[30]</code>
Heap(tas)	Taille variable, dépend du dev et de la ram disponible	<code>int* etud = (int) malloc (5*sizeof(int))</code>
malloc	Réserve un nombre d'octet brut défini par le programme	<code>int* etud = (int) malloc (5*sizeof(int))</code>
free	Permet de libérer l'espace pris par un objet créé par malloc	<code>free(etud)</code>
Memory leak	Fuite de mémoire qui intervient lorsque un objet n'a pas été supprimé avec un free	<code>int* etud = (int) malloc (5*sizeof(int))</code> (pas de free inséré)
Dangling pointer	Pointeur qui pointe vers une mémoire qui n'existe plus, c'est le cas pour la donnée que cette variable renvoie	<code>Free(etud)</code> <code>etud.getNom();</code>
Buffer overflow	Erreur critique, arrive quand le programme écrit au-delà du mémoire alloué à la ressource	(pas de free inséré)
Classe	Objet qui contient méthodes (fonctions) et attributs (valeurs) très pratique pour structurer des données	<code>Class Etudiant{</code> <code>Public:</code> <code>Char nom[10];</code> <code>Char getNom();}</code>
Constructeur	Permet d'initialiser les valeurs d'une classe	<code>Etudiant(Char a, int ag):</code> <code>nom(a), age(ag);</code>
Encapsulation	Principe de cacher des données pour ne pas directement pouvoir modifier des valeurs	<code>Private:</code> <code>Char nom[10]</code> <code>Int Age ;</code>

